

17

File Processing

OBJECTIVES

In this chapter you will learn:

- To create, read, write and update files.
- Sequential file processing.
- Random-access file processing.
- To use high-performance unformatted I/O operations.
- The differences between formatted-data and raw-data file processing.
- To build a transaction-processing program using random-access file processing.

17.2 The Data Hierarchy

- **File**

- **Composed of a group of related records**
 - **Example: A payroll file containing one record for each employee.**

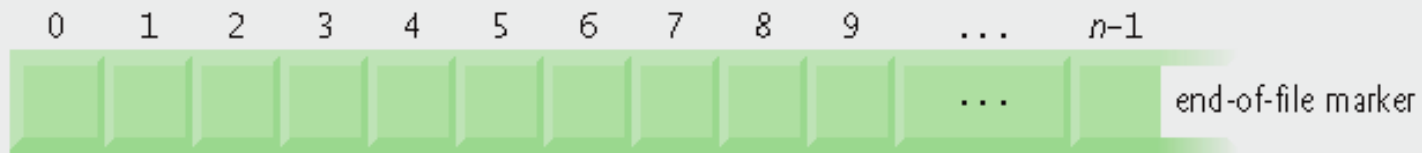
- **Database**

- **Composed of a group of related files**
- **Managed by a collection of programs called a database management system (DBMS)**

17.3 Files and Streams

- **Files**

- Viewed by C++ as a sequence of bytes
- Ends either with an end-of-file marker or at a system-recorded byte number.



17.3 Files and Streams

- **Files**

- Communication between a program and a file is performed through stream objects.
 - `<fstream>` header file
 - Stream class templates
 - `basic_ifstream` – for file input
 - `basic_ofstream` – for file output
 - `basic_fstream` – for file input and output

17.4 Creating a Sequential File

- **Sequential file: a common type to organize records.**
- **File structure**
 - **The programmer must structure sequential files.**
 - **C++ does not impose structures on sequential files.**

17.4 Creating a Sequential File (Cont.)

- **Example: Create a sequential file and write data into the file.**

Outline

Fig17_04.cpp

(1 of 2)

```
1 // Fig. 17.4: Fig17_04.cpp
2 // Create a sequential file.
3 #include <iostream>
4 using std::cerr;
5 using std::cin;
6 using std::cout;
7 using std::endl;
8 using std::ios;
9
10 #include <fstream> // file stream
11 using std::ofstream; // output file stream
12
13 #include <cstdlib>
14 using std::exit; // exit function prototype
15
16 int main()
17 {
18     // ofstream constructor opens file
19     ofstream outClientFile( "clients.dat", ios::out );
20
21     // exit program if unable to create file
22     if ( !outClientFile ) // overloaded ! operator
23     {
24         cerr << "File could not be opened" << endl;
25         exit( 1 );
26     } // end if
27
28     cout << "Enter the account, name, and balance." << endl
29         << "Enter end-of-file to end input.\n? ";
```

```
30
31  int account;
32  char name[ 30 ];
33  double balance;
34
35  // read account, name and balance from cin, then place in file
36  while ( cin >> account >> name >> balance )
37  {
38      outClientFile << account << " " << name << " " << balance << endl;
39      cout << "? ";
40  } // end while
41
42  return 0; // ofstream destructor closes file
43 } // end main
```

```
Enter the account, name, and balance.
Enter end-of-file to end input.
? 100 Jones 24.98
? 200 Doe 345.67
? 300 White 0.00
? 400 Stone -42.16
? 500 Rich 224.62
? ^Z
```

17.4 Creating a Sequential File (Cont.)

```
Line 18: ofstream outClientFile( "clients.dat", ios::out );
```

- **Creating an `ofstream` object**
 - Opens a file for output.
 - Constructor takes two arguments.
 - **Filename:** If the file does not exist, it will be created using that file name.
 - **File-open mode**
 - `ios::out` – the default mode
 - Overwrites preexisting data in the file.
 - `ios::app`
 - Appends data to the end of the file.

17.4 Creating a Sequential File (Cont.)

Line 22: `if ( !outClientFile )`

- **`ios` operators (usable with `ofstream`)**
 - **Operator member function `operator!`**
 - Returns `true` if either the `failbit` or `badbit` is set.
 - **Operator member function `operator void *`**
 - Converts the stream to a pointer.
 - Returns the null pointer if either the `failbit` or `badbit` is set.

17.4 Creating a Sequential File (Cont.)

```
Line 38: outClientFile << account << ' ' << name << ' ' << balance << endl;
```

- **Write a set of data (records) to the file `clients.dat`, using the stream insertion operator `<<`.**

17.4 Creating a Sequential File (Cont.)

- **When the program finishes, it invokes the `outClientFile` object's destructor function implicitly, which closes the `clients.dat` file.**
 - **Constructor:** it is called when execution reaches the point where an object is created.
 - **Destructor:** it is called when execution leaves the object's scope.
 - It is not called if the program terminates with a call to function `exit` or `abort`.

17.4 Creating a Sequential File (Cont.)

- The programmer can close the file explicitly using member function `close`.

```
outClientFile.close();
```

17.5 Reading Data from a Sequential File

- **Example: Open a sequential file and read the data from the file.**

```

1 // Fig. 17.7: Fig17_07.cpp
2 // Reading and printing a sequential file.
3 #include <iostream>
4 using std::cerr;
5 using std::cout;
6 using std::endl;
7 using std::fixed;
8 using std::ios;
9 using std::left;
10 using std::right;
11 using std::showpoint;
12
13 #include <fstream> // file stream
14 using std::ifstream; // input file stream
15
16 #include <iomanip>
17 using std::setw;
18 using std::setprecision;
19
20 #include <string>
21 using std::string;
22
23 #include <cstdlib>
24 using std::exit; // exit function prototype
25
26 void outputLine( int, const string, double ); // prototype

```

Outline

Fig17_07.cpp

(1 of 3)

This program requires a file “credit.dat” created from Fig17_04.cpp. Put it in the same program folder.

```
27
28 int main()
29 {
30     // ifstream constructor opens the file
31     ifstream inClientFile( "clients.dat", ios::in );
32
33     // exit program if ifstream could not open file
34     if ( !inClientFile )
35     {
36         cerr << "File could not be opened" << endl;
37         exit( 1 );
38     } // end if
39
40     int account;
41     char name[ 30 ];
42     double balance;
43
44     cout << left << setw( 10 ) << "Account" << setw( 13 )
45         << "Name" << "Balance" << endl << fixed << showpoint;
46
47     // display each record in file
48     while ( inClientFile >> account >> name >> balance )
49         outputLine( account, name, balance );
50
51     return 0; // ifstream destructor closes the file
52 } // end main
```

```
53
54 // display single record from file
55 void outputLine( int account, const string name, double balance )
56 {
57     cout << left << setw( 10 ) << account << setw( 13 ) << name
58         << setw( 7 ) << setprecision( 2 ) << right << balance << endl;
59 } // end function outputLine
```

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

Outline

Fig17_07.cpp

(3 of 3)

17.5 Reading Data from a Sequential File

```
Line 31: ifstream inClientFile( "clients.dat", ios::in );
```

- **Creating an `ifstream` object**
 - Opens a file for input.
 - Constructor takes two arguments
 - A filename
 - A file-open mode
 - `ios::in` – the default mode
 - Can only read from the file

17.5 Reading Data from a Sequential File

```
Line 48: while ( inClientFile >> account >> name >> balance )
```

- **Read a set of data (records) from the file.**
 - Each time this statement executes, it reads another record from the file.
 - When the end of the file has been reached, the implicit call returns the null pointer (which converts to the boolean value `false`), the `ifstream` destructor function closes the file and the program terminates.

17.5 Reading Data from a Sequential File (Cont.)

- **File-position pointer**

- The byte number of the next byte to be read or written
- Usually programs start reading from the beginning of the file and read all the data consecutively until the desired data is found.
- To get and change the file-position pointer, we have **seekg** (“seek get”) and **seekp** (“seek put”).
 - **Get:** the byte number in the file from the next input is to occur.
 - **Put:** the byte number in the file from the next output should be placed.

17.5 Reading Data from a Sequential File (Cont.)

- Member functions **seekg** and **seekp** (of **istream** and **ostream**, respectively)
 - Repositions the file-position pointer to the specified location.
 - A second argument can specify the seek direction.
 - **ios::beg** – the default
 - Positioning relative to the beginning
 - **ios::cur**
 - Positioning relative to the current position
 - **ios::end**
 - Positioning relative to the end

17.5 Reading Data from a Sequential File (Cont.)

- `fileObject.seekg( n );`
 - Position to the *n*th byte of `fileObject`
- `fileObject.seekg( n, ios::cur );`
 - Position *n* bytes forward in `fileObject`
- `fileObject.seekg( n, ios::end );`
 - Position *n* bytes back from end of `fileObject`
- `fileObject.seekg( 0, ios::end );`
 - Position at end of `fileObject`

17.5 Reading Data from a Sequential File (Cont.)

- **File-position pointer (Cont.)**
 - Member functions `tellg` and `tellp` (of `istream` and `ostream`, respectively)
 - Returns current position of the file-position pointer as type `long`.
 - Example
 - `Location = fileObject.tellg();`

17.5 Reading Data from a Sequential File (Cont.)

- **Example: Use `seekg` to reposition the file-position pointer.**

```

1 // Fig. 17.8: Fig17_08.cpp
2 // Credit inquiry program.
3 #include <iostream>
4 using std::cerr;
5 using std::cin;
6 using std::cout;
7 using std::endl;
8 using std::fixed;
9 using std::ios;
10 using std::left;
11 using std::right;
12 using std::showpoint;
13
14 #include <fstream>
15 using std::ifstream;
16
17 #include <iomanip>
18 using std::setw;
19 using std::setprecision;
20
21 #include <string>
22 using std::string;
23
24 #include <cstdlib>
25 using std::exit; // exit function prototype
26
27 enum RequestType { ZERO_BALANCE = 1, CREDIT_BALANCE, DEBIT_BALANCE, END };
28 int getRequest();
29 bool shouldDisplay( int, double );
30 void outputLine( int, const string, double );

```

Outline

Fig17_08.cpp

(1 of 6)

*Refer to Chapter 6.8 for
the usage of “**enum**”*

```
31
32 int main()
33 {
34     // ifstream constructor opens the file
35     ifstream inClientFile( "clients.dat", ios::in );
36
37     // exit program if ifstream could not open file
38     if ( !inClientFile )
39     {
40         cerr << "File could not be opened" << endl;
41         exit( 1 );
42     } // end if
43
44     int request;
45     int account;
46     char name[ 30 ];
47     double balance;
48
49     // get user's request (e.g., zero, credit or debit balance)
50     request = getRequest();
51
```

```

52 // process user's request
53 while ( request != END )
54 {
55     switch ( request )
56     {
57         case ZERO_BALANCE:
58             cout << "\nAccounts with zero balances:\n";
59             break;
60         case CREDIT_BALANCE:
61             cout << "\nAccounts with credit balances:\n";
62             break;
63         case DEBIT_BALANCE:
64             cout << "\nAccounts with debit balances:\n";
65             break;
66     } // end switch
67
68     // read account, name and balance from file
69     inClientFile >> account >> name >> balance;
70
71     // display file contents (until eof)
72     while ( !inClientFile.eof() )
73     {
74         // display record
75         if ( shouldDisplay( request, balance ) )
76             outputLine( account, name, balance );
77
78         // read account, name and balance from file
79         inClientFile >> account >> name >> balance;
80     } // end inner while
81

```

Outline

Fig17_08.cpp

(3 of 6)

```

82     inClientFile.clear(); // reset eof for next input
83     inClientFile.seekg( 0 ); // reposition to beginning of file
84     request = getRequest(); // get additional request from user
85 } // end outer while
86
87     cout << "End of run." << endl;
88     return 0; // ifstream destructor closes the file
89 } // end main
90
91 // obtain request from user
92 int getRequest()
93 {
94     int request; // request from user
95
96     // display request options
97     cout << "\nEnter request" << endl
98         << " 1 - List accounts with zero balances" << endl
99         << " 2 - List accounts with credit balances" << endl
100        << " 3 - List accounts with debit balances" << endl
101        << " 4 - End of run" << fixed << showpoint;
102
103     do // input user request
104     {
105         cout << "\n? ";
106         cin >> request;
107     } while ( request < ZERO_BALANCE && request > END );
108
109     return request;
110 } // end function getRequest
111

```

Outline

Fig17_08.cpp

(4 of 6)

```

112// determine whether to display given record
113bool shouldDisplay( int type, double balance )
114{
115    // determine whether to display zero balances
116    if ( type == ZERO_BALANCE && balance == 0 )
117        return true;
118
119    // determine whether to display credit balances
120    if ( type == CREDIT_BALANCE && balance < 0 )
121        return true;
122
123    // determine whether to display debit balances
124    if ( type == DEBIT_BALANCE && balance > 0 )
125        return true;
126
127    return false;
128} // end function shouldDisplay
129
130// display single record from file
131void outputLine( int account, const string name, double balance )
132{
133    cout << left << setw( 10 ) << account << setw( 13 ) << name
134        << setw( 7 ) << setprecision( 2 ) << right << balance << endl;
135} // end function outputLine

```

Outline

Fig17_08.cpp

(5 of 6)

Outline

Fig17_08.cpp

(6 of 6)

```
Enter request
1 - List accounts with zero balances
2 - List accounts with credit balances
3 - List accounts with debit balances
4 - End of run
? 1

Accounts with zero balances:
300      White      0.00

Enter request
1 - List accounts with zero balances
2 - List accounts with credit balances
3 - List accounts with debit balances
4 - End of run
? 2

Accounts with credit balances:
400      Stone     -42.16

Enter request
1 - List accounts with zero balances
2 - List accounts with credit balances
3 - List accounts with debit balances
4 - End of run
? 3

Accounts with debit balances:
100      Jones      24.98
200      Doe        345.67
500      Rich       224.62

Enter request
1 - List accounts with zero balances
2 - List accounts with credit balances
3 - List accounts with debit balances
4 - End of run
? 4

End of run.
```

17.6 Updating Sequential Files

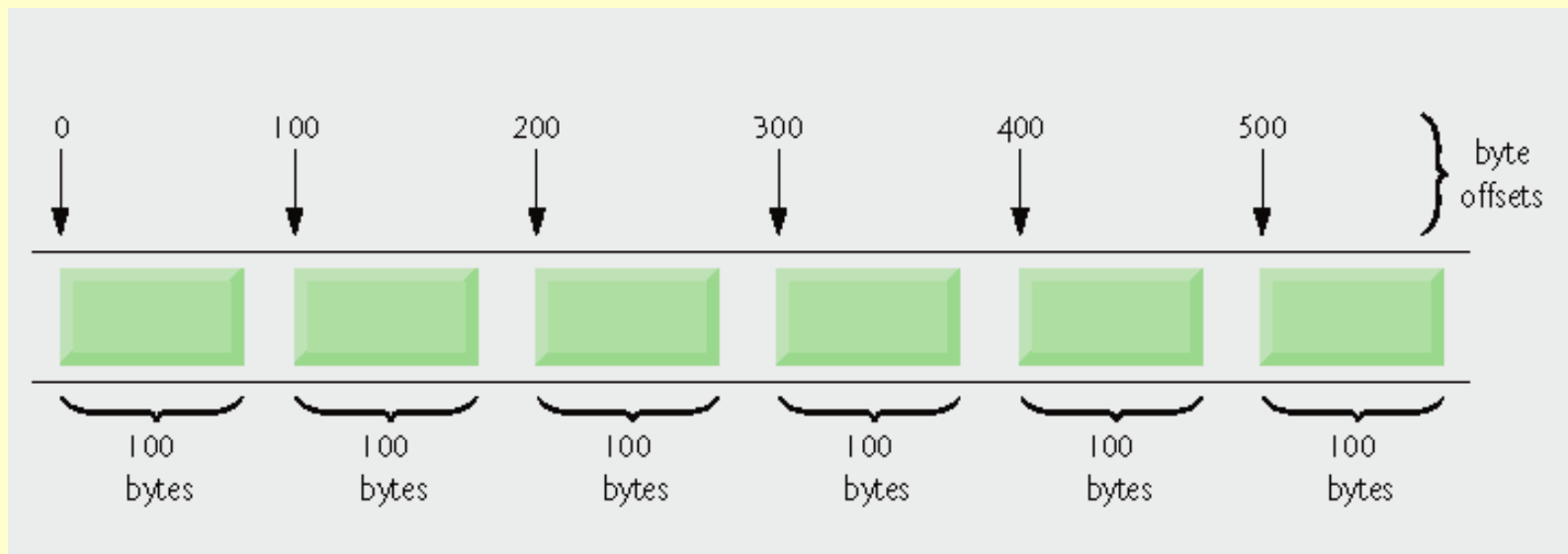
- **Updating a record in a sequential file**
 - **The new record could be longer than the old record.**
 - **If it is, it could overwrite the next sequential record.**
 - **You would have to rewrite every record into another file.**
 - **Copy over all records before this one.**
 - **Write new version of this record.**
 - **Copy over all records after this one.**
 - **This might be acceptable if you are updating many records.**

17.7 Random-Access Files

- **Random-access files**
 - **Necessary for instant-access applications**
 - **Instant-access application: a particular record must be located immediately.**
 - **Example: airline reservation system**
 - **A record can be inserted, deleted or modified without affecting other records.**
 - **Various techniques can be used.**
 - **Require that all records be of the same length, arranged in the order of the record keys.**
 - **Program can calculate the exact location of any record based on the record size and record key.**

17.7 Random-Access Files

- **C++ view of a random-access file**



17.8 Creating a Random-Access File

- **Example: Create a random-access file and output 100 blank records into the file.**

```

1 // Fig. 17.10: ClientData.h
2 // Class ClientData definition used in Fig. 17.12–Fig. 17.15.
3 #ifndef CLIENTDATA_H
4 #define CLIENTDATA_H
5
6 #include <string>
7 using std::string;
8
9 class ClientData
10 {
11 public:
12     // default ClientData constructor
13     ClientData( int = 0, string = "", string = "", double = 0.0 );
14
15     // accessor functions for accountNumber
16     void setAccountNumber( int );
17     int getAccountNumber() const;
18
19     // accessor functions for lastName
20     void setLastName( string );
21     string getLastName() const;
22
23     // accessor functions for firstName
24     void setFirstName( string );
25     string getFirstName() const;

```

Outline

ClientData.h

(1 of 2)

```
26
27 // accessor functions for balance
28 void setBalance( double );
29 double getBalance() const;
30 private:
31 int accountNumber;
32 char lastName[ 15 ];
33 char firstName[ 10 ];
34 double balance;
35 }; // end class ClientData
36
37 #endif
```

Outline

ClientData.h

(2 of 2)

```

1 // Fig. 17.11: ClientData.cpp
2 // Class ClientData stores customer's credit information.
3 #include <string>
4 using std::string;
5
6 #include "ClientData.h"
7
8 // default ClientData constructor
9 ClientData::ClientData( int accountNumberValue,
10     string lastNameValue, string firstNameValue, double balanceValue )
11 {
12     setAccountNumber( accountNumberValue );
13     setLastName( lastNameValue );
14     setFirstName( firstNameValue );
15     setBalance( balanceValue );
16 } // end ClientData constructor
17
18 // get account-number value
19 int ClientData::getAccountNumber() const
20 {
21     return accountNumber;
22 } // end function getAccountNumber
23
24 // set account-number value
25 void ClientData::setAccountNumber( int accountNumberValue )
26 {
27     accountNumber = accountNumberValue; // should validate
28 } // end function setAccountNumber

```

Outline

ClientData.cpp

(1 of 3)

```

29
30 // get last-name value
31 string ClientData::getLastName() const
32 {
33     return lastName;
34 } // end function getLastName
35
36 // set last-name value
37 void ClientData::setLastName( string lastNameString )
38 {
39     // copy at most 15 characters from string to lastName
40     const char *lastNameValue = lastNameString.data();
41     int length = lastNameString.size();
42     length = ( length < 15 ? length : 14 );
43     strncpy_s( lastName, lastNameValue, length );
44
45 } // end function setLastName
46
47 // get first-name value
48 string ClientData::getFirstName() const
49 {
50     return firstName;
51 } // end function getFirstName

```

Outline

ClientData.cpp

(2 of 3)

17.8 Creating a Random-Access File (Cont.)

```
Line 40: const char *lastNameValue = lastNameString.data();
```

- **The member function `data` returns an array containing the characters of the `string`.**

17.8 Creating a Random-Access File (Cont.)

```
Line 41: int length = lastNameString.size();
```

- The member function **size** gets the length of **lastNameString**.

```

52
53 // set first-name value
54 void ClientData::setFirstName( string firstNameString )
55 {
56     // copy at most 10 characters from string to firstName
57     const char *firstNameValue = firstNameString.data();
58     int length = firstNameString.size();
59     length = ( length < 10 ? length : 9 );
60     strncpy_s( firstName, firstNameValue, length );
61
62 } // end function setFirstName
63
64 // get balance value
65 double ClientData::getBalance() const
66 {
67     return balance;
68 } // end function getBalance
69
70 // set balance value
71 void ClientData::setBalance( double balanceValue )
72 {
73     balance = balanceValue;
74 } // end function setBalance

```

Outline

ClientData.cpp

(3 of 3)

```
1 // Fig. 17.12: Fig17_12.cpp
2 // Creating a randomly accessed file.
3 #include <iostream>
4 using std::cerr;
5 using std::endl;
6 using std::ios;
7
8 #include <fstream>
9 using std::ofstream;
10
11 #include <cstdlib>
12 using std::exit; // exit function prototype
13
14 #include "ClientData.h" // ClientData class definition
15
```

Outline

Fig17_12.cpp

(1 of 2)

```

16 int main()
17 {
18     ofstream outCredit( "credit.dat", ios::binary );
19
20     // exit program if ofstream could not open file
21     if ( !outCredit )
22     {
23         cerr << "File could not be opened." << endl;
24         exit( 1 );
25     } // end if
26
27     ClientData blankClient; // constructor zeros out each data member
28
29     // output 100 blank records to file
30     for ( int i = 0; i < 100; i++ )
31         outCredit.write( reinterpret_cast< const char * >( &blankClient ),
32                         sizeof( ClientData ) );
33
34     return 0;
35 } // end main

```

Outline

Fig17_12.cpp

(2 of 2)

17.8 Creating a Random-Access File

```
Line 18: ofstream outCredit( "credit.dat", ios::binary );
```

- **Open the file for output in binary mode (i.e., random-access file).**
 - Sequential files are in text mode (i.e., all data are texts).

17.8 Creating a Random-Access File

```
Line 31: outCredit.write( reinterpret_cast< const char * >( &blankClient ),  
Line 32:      sizeof( ClientData ) );
```

- **ostream member function write**
 - Writes a number of bytes from a location in memory to the stream.
 - First argument
 - A **const char *** pointing to bytes in memory
 - Second argument
 - A **size_t** specifying the number of bytes to write

17.8 Creating a Random-Access File (Cont.)

- **Operator `reinterpret_cast`**
 - Converts `&blankClient` to `const char *`.
 - Casts a pointer of one type to an unrelated type.
 - Is performed at compile time.

17.9 Writing Data Randomly to a Random-Access File

- **Example: Write data to a random-access file.**

```

1 // Fig. 17.13: Fig17_13.cpp
2 // Writing to a random-access file.
3 #include <iostream>
4 using std::cerr;
5 using std::cin;
6 using std::cout;
7 using std::endl;
8 using std::ios;
9
10 #include <iomanip>
11 using std::setw;
12
13 #include <fstream>
14 using std::fstream;
15
16 #include <cstdlib>
17 using std::exit; // exit function prototype
18
19 #include "ClientData.h" // ClientData class definition
20
21 int main()
22 {
23     int accountNumber;
24     char lastName[ 15 ];
25     char firstName[ 10 ];
26     double balance;
27
28     fstream outCredit( "credit.dat", ios::in | ios::out | ios::binary );
29

```

Outline

Fig17_13.cpp

(1 of 4)

This program requires a file “credit.dat” created from Fig17_12.cpp. Put it in the same program folder.

```

30 // exit program if fstream cannot open file
31 if ( !outCredit )
32 {
33     cerr << "File could not be opened." << endl;
34     exit( 1 );
35 } // end if
36
37 cout << "Enter account number (1 to 100, 0 to end input)\n? ";
38
39 // require user to specify account number
40 ClientData client;
41 cin >> accountNumber;
42
43 // user enters information, which is copied into file
44 while ( accountNumber > 0 && accountNumber <= 100 )
45 {
46     // user enters last name, first name and balance
47     cout << "Enter lastname, firstname, balance\n? ";
48     cin >> setw( 15 ) >> lastName;
49     cin >> setw( 10 ) >> firstName;
50     cin >> balance;
51
52     // set record accountNumber, lastName, firstName and balance values
53     client.setAccountNumber( accountNumber );
54     client.setLastName( lastName );
55     client.setFirstName( firstName );
56     client.setBalance( balance );
57

```

Outline

Fig17_13.cpp

(2 of 4)

```

58 // seek position in file of user-specified record
59 outCredit.seekp( ( client.getAccountNumber() - 1 ) *
60     sizeof( ClientData ) );
61
62 // write user-specified information in file
63 outCredit.write( reinterpret_cast< const char * >( &client ),
64     sizeof( ClientData ) );
65
66 // enable user to enter another account
67 cout << "Enter account number\n? ";
68 cin >> accountNumber;
69 } // end while
70
71 return 0;
72 } // end main

```

Outline

Fig17_13.cpp

(3 of 4)

Outline

Fig17_13.cpp

(4 of 4)

Enter account number (1 to 100, 0 to end input)

? 37

Enter lastname, firstname, balance

? Barker Doug 0.00

Enter account number

? 29

Enter lastname, firstname, balance

? Brown Nancy -24.54

Enter account number

? 96

Enter lastname, firstname, balance

? Stone Sam 34.98

Enter account number

? 88

Enter lastname, firstname, balance

? Smith Dave 258.34

Enter account number

? 33

Enter lastname, firstname, balance

? Dunn Stacey 314.33

Enter account number

? 0

17.9 Writing Data Randomly to a Random-Access File

```
Line 28: fstream outCredit( "credit.dat", ios::in | ios::out | ios:: binary );
```

- **Opening for input and output in binary mode**
 - Use an `fstream` object.
 - Combine file-open modes `ios::in`, `ios::out` and `ios::binary`.
 - Separate each open mode from the next with the bitwise inclusive OR operator (`|`).

17.9 Writing Data Randomly to a Random-Access File

```
Line 59: outCredit.seekp( ( client.getAccountNumber() - 1 ) *  
Line 60:     sizeof( ClientData ) );
```

- Use function **seekp** to set the “put” file-position pointer to the specific position.
 - Example calculation
 - $(n - 1) * \text{sizeof}(\text{ClientData})$
 - Byte location for *n*th **ClientData** record

17.10 Reading from a Random-Access File Sequentially

- **Example: Sequentially read a random-access file.**

```

1 // Fig. 17.14: Fig17_14.cpp
2 // Reading a random access file sequentially.
3 #include <iostream>
4 using std::cerr;
5 using std::cout;
6 using std::endl;
7 using std::fixed;
8 using std::ios;
9 using std::left;
10 using std::right;
11 using std::showpoint;
12
13 #include <iomanip>
14 using std::setprecision;
15 using std::setw;
16
17 #include <fstream>
18 using std::ifstream;
19 using std::ofstream;
20
21 #include <cstdlib>
22 using std::exit; // exit function prototype
23
24 #include "ClientData.h" // ClientData class definition
25
26 void outputLine( ostream&, const ClientData & ); // prototype
27

```

Outline

Fig17_14.cpp

(1 of 3)

```

28 int main()
29 {
30     ifstream inCredit( "credit.dat", ios::in );
31
32     // exit program if ifstream cannot open file
33     if ( !inCredit )
34     {
35         cerr << "File could not be opened." << endl;
36         exit( 1 );
37     } // end if
38
39     cout << left << setw( 10 ) << "Account" << setw( 16 )
40         << "Last Name" << setw( 11 ) << "First Name" << left
41         << setw( 10 ) << right << "Balance" << endl;
42
43     ClientData client; // create record
44
45     // read first record from file
46     inCredit.read( reinterpret_cast< char * >( &client ),
47         sizeof( ClientData ) );
48
49     // read all records from file
50     while ( inCredit && !inCredit.eof() )
51     {
52         // display record
53         if ( client.getAccountNumber() != 0 )
54             outputLine( cout, client );
55

```

Outline

Fig17_14.cpp

(2 of 3)

```

56 // read next from file
57 inCredit.read( reinterpret_cast< char * >( &client ),
58             sizeof( ClientData ) );
59 } // end while
60
61 return 0;
62 } // end main
63
64 // display single record
65 void outputLine( ostream &output, const ClientData &record )
66 {
67     output << left << setw( 10 ) << record.getAccountNumber()
68         << setw( 16 ) << record.lastName()
69         << setw( 11 ) << record.firstName()
70         << setw( 10 ) << setprecision( 2 ) << right << fixed
71         << showpoint << record.getBalance() << endl;
72 } // end function outputLine

```

Account	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

Outline

Fig17_14.cpp

(3 of 3)

17.10 Reading from a Random-Access File Sequentially

```
Line 46: inCredit.read( reinterpret_cast< char * >( &Client ),  
Line 47:     sizeof( ClientData ) );
```

- **ifstream member function read**
 - Inputs a number of bytes from the current file position in the stream into an object.
 - First argument
 - A **char *** pointing to the object in memory
 - Second argument
 - A **size_t** specifying the number of bytes to input

Until Next Time

- **Review Chapter 17**
- **Study the bonus example *Fig17_15.cpp* from the course website**
- **Preview GUI Lecture Notes**